Štefan Prokop

Bezpečný kód



10 kroků pro bezpečné, škálovatelné, spolehlivé a snadno použitelné API - bez nutnosti programování

Příručka pro IT manažery

Cílem nás všech je při tvorbě API a software zajistit co největší bezpečnost a kvalitu. Často ale selháváme. Funkcionality přepisujeme, měníme architekturu API, potýkáme se s výkonnostními problémy, řešíme časté dotazy na to, jak API funguje, atd. Důvodů může být samozřejmě víc, ale zaměřím se na ty, se kterými se v praxi setkávám dennodenně a považuju je za nejčastější viníky. Jsem si jistý, že bezpečnost a kvalita nejen API, ale jakéhokoli software, může být výrazně zlepšena pomocí následujících 10 kroků.

Tohle je hlavní důvod, proč máte problémy s API

API je pro většinu firem takový kostlivec ve skříni. A vsadím se, že se v následujícím popisu více či méně poznáte i vy. Nejspíš je vám jasné, co má vaše aplikace dělat, o tom nikdo ve firmě nepochybuje. Možná už ale nevíte, jaké nároky na API vlastně máte. Má být schopné obsloužit 100.000 požadavků za sekundu? Musí API dodržovat PCI DSS nebo jinou bezpečnostní regulaci? Musíte zajistit nějakou dostupnost podle SLA? Je potřeba, aby se koncovým uživatelům s API pracovalo dobře a jednoduše? Nebo jen děláte prototyp a tohle všechno je vám jedno, protože víte, že budete API přepisovat hned, jak získáte investici? Těch otázek je mnoho. A každá z nich vám pomáhá udělat ten nejzásadnější krok při vývoji API – určit si jeho **cíl**.

API je samostatný projekt; i přesto, že jde o část většího celku samotné aplikace. Jen a pouze na základě jasného cíle a očekávání se může vývojový tým zaměřit na to, co je opravdu důležité. Členové týmu to vezmou v potaz během činností, jako je modelování hrozeb, návrh architektury, plánování sprintů a práce atd.

Pokud nemáte jasný cíl, nejspíš se stane to, že budete tříštit pozornost při vývoji. Budete se věnovat úkolům, které mohou počkat, nebo které dokonce naruší nějaké vaše nevyřčené představy o fungování API. Například máte v hlavě myšlenku, jak je vaše API rychlé, ale implementací analytického nástroje ho značně zpomalíte. Rázem řešíte, co s tím, abyste ho opět zrychlili.

Cíl by měl být znám celému vývojovému týmu. Od byznys ownera, projekťáka, přes vývojáře, externisty, UX / UI až po testery. Celý tým musí být na jedné vlně a musí vědět, co od aplikace a API očekáváte a kde můžete dělat kompromisy. Firmy často chtějí rychlé API, ale pak se stane, že UXák navrhne tak “fancy” funkcionalitu, že je rychlost rázem v tahu., Jak je to možné? A jak je možné, že to vůbec projde? Odpověď na obě otázky se ukrývá právě v chybějícím cíli. UXák neví, že má aplikace být primárně rychlá, spíš než vymazlená. A nikdo z vývojového týmu se neozve, protože neví, na co se má soustředit – neví, co je ten cíl.

Problém je způsobený tím, že API je velmi abstraktní. Chybí tam UI, frontend, pomocí kterého s ním interagujete. Je tedy těžké si ho nějak hmatatelně představit. Byznys owner nebo projekťák řeší aplikaci jako celek a věnují se hlavně frontendu, protože je “hmatatelný”. Specifikace je vytvářena pro frontend a API se implementuje právě podle UI specifikace, nebo jdou ruku v ruce. Mělo by to být spíš naopak, v každém případě neuškodí, když se o tom v týmu budete víc bavit. API je důvod, proč daný projekt ve výsledku funguje, a proto by měl mít vlastní cíl a hlavní pozornost.

Důležité je si uvědomit, že nikdy nebudete mít všechno - nebudete mít rychlé a zároveň super bezpečné API, případně uživatelsky přívětivé atd. Některé požadavky se vzájemně vylučují a bude potřeba dělat nějaké kompromisy. Proto je potřeba, aby to řešil celý tým a všichni měli stejné (a kompletní) informace. Díky tomu se jejich činy a úkoly vzájemně nevyrušují. Důsledkem chybějících cílů také často bývá nedodržení termínu dodání a překročení rozpočtu.

Krok 1: Určete cíle svého API

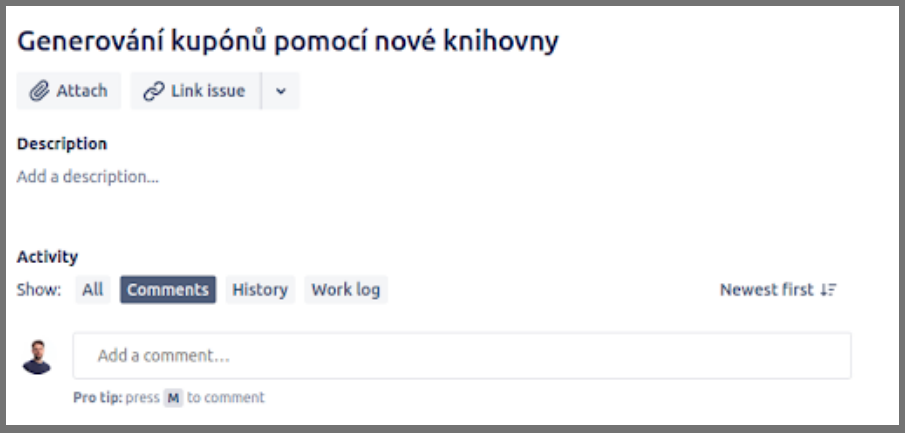
Jak na API cíle?

- Definujte primární účel API a scope.
- Identifikujte specifické funkcionality, které by API mělo poskytovat (ty nejdůležitější pro váš byznys).
- Definujte cílovku a jejich možné požadavky na API.
- Určete očekávané nároky na provoz (počet requestů…).
- Identifikujte potenciální slabá místa z hlediska výkonu.
- Promyslete potenciální strategii škálování.
- Nastavte jasné metriky týkající se výkonu API (doba odezvy, propustnost, počet souběžných requestů…).
- Definujte autentizační a autorizační mechanismus.
- Nastavte cíle (code coverage, regrese) a strategie (unit, integration, end-to-end) v oblasti testování.
- Zajistěte pravidelnou týmovou komunikaci. Bavte se s týmem s o aktuálních cílech a jejich případných změnách.
- Nenavrhujte frontendovou funkcionalitu bez přítomnosti API vývojáře.
- Pravidelně monitorujte a vyhodnocujte stanovené metriky.
- Chvalte své zaměstnance při jakémkoli progresu a přiblížení se k cíli. Zvyšujete tím jejich motivaci a zajišťujete, že se budou soustředit na ty správné věci a činnosti.

„Týmová komunikace je u nás v pořádku…" Víte to jistě?

V praxi si často všímám, že CTO, SW architekt nebo teamleader znají cíl jejich API a celého produktu perfektně. Problém nastává, pokud se na něj zeptám řadového člena týmu, který má na starosti implementaci, testování nebo další činnosti. K těmto lidem se většinou konkrétní informace nedostanou, nikdo jim o projektu víc neřekl. Spoléhají se na to, že to bude napsané v nástroji pro správu úkolů (Jira, Trello, atd.) nebo jim to někdo pošle v e-mailu, případně se to bude řešit na schůzce. Co když to tak ale není? Proto je důležité, aby byl celý tým na stejné vlně.

Tento problém snadno řeší již zmíněné nástroje pro správu úkolů. Často jsou ale mylně interpretovány. Podle většiny lidí jde jen o nějaký záznam toho, na čem se zrovna pracuje (nebo že se vůbec na něčem pracuje) a slouží ke sledování produktivity zaměstnanců a měření času stráveného na daném úkolu. To je naprosté nepochopení, protože pak zadání vypadají tak, jako je to na obrázku.



Představte si, že jste dostali takové zadání. Teď mi řekněte krok za krokem, jak budete postupovat. Nemusíte být vývojář a znát jakékoli programovací jazyky. Prostě mi řekněte, jaký bude postup - analyticky, teoreticky. Teď si představte, že jste tester. Vaší povinností je zaručit, že je funkcionalita naimplementována správně. Spousta testerů má přímou odpovědnost za to, jak daná funkcionalita funguje. Už cítíte to nebezpečí? Co kdybych vám pak strhnul 10 % z platu, protože klient nahlásil bug, který jste měli odhalit v rámci testování této funkcionality?

Zadání je projektová dokumentace. Každá funkcionalita by měla být podrobně popsána a mělo by být jasné, co má kdo udělat a jak jednoznačně ověřit, že to dotyčný opravdu udělal.

Většina firem se spoléhá na mítinky, kde si tyto informace řeknou. Dokážete mi teď z hlavy popsat, co jste řešili a na čem jste se domluvili na callu s klientem před 3 týdny? Sotva mítink skončí, zaměstnanci si pamatují minimum z toho, co se tam probíralo, zejména pokud to byl nudný a monotónní mítink. Pravděpodobnost, že pak bude zaměstnanec znát přesný popis zadání, se blíží nule.

Je povinnost projektového manažera nebo tým lídra, v závislosti na tom, jak to máte nastavené, aby dal dohromady kvalitní zadání. Pokud zadání vypadá jako na obrázku, dělá dotyčný špatně svou práci. V takovém případě musíte určit, proč tomu tak je. Má toho hodně a nestíhá? Pak se o to musí postarat jeho nadřízený. Prostě to jen nedělá? Vyhoďte ho. Stojí vás víc, než mu platíte.

Pokud jako pracovník dostanu takové zadání, první, co udělám, je, že se dotyčného ptám, co mám vlastně dělat.

Na to se ho pak bude ptát každý další pracovník, který s tímto zadáním přijde do styku. Snadno se tedy stane, že projekťák stráví 8 hodin čistého času vysvětlováním jedné funkcionality, kterou mohl během hodiny popsat. Stejný počet hodin na tom pak také stráví dohromady jednotliví pracovníci. Pokud čas všech zúčastněných převedeme na peníze, dostaneme se k tomu, že by se vám vzhledem k hodinovkám vyplatilo najmout člověka, který ta zadání bude sepisovat.

Projekťáci mají velký vliv i na bezpečnost. Právě oni totiž komunikují se zákazníkem a je jejich úkolem zjistit, co mají vývojáři zabezpečit. Vývojář nemá šanci určit, že v tabulce budete zobrazovat firemní know-how, které nesmí uniknout. Pro něj je to jen tabulka. Někdo mu musí říct, že je to velmi citlivá informace, která musí být vhodně zabezpečená. Ano, něco dokáže odhadnout, ale o regulacích, certifikacích a dalších věcech nemá ani ponětí, není to právník. Někdo mu to musí říct.

Všechny tyto informace se nachází se vaší projektové dokumentaci, která odpovídá seznamu úkolů. Pokud z firmy odejde klíčový člověk nebo přijde nový, jedete v klidu dál, protože máte vše podrobně sepsané a víte, co máte dělat. Jde o jeden z hlavních důvodů, proč nejste schopni dosáhnout vašich odhadů a deadlinů. A to ještě neřešíme remote work, který je dnes na vzestupu. Díky chybějící projektové dokumentaci a nedostatečnému zadání je projekt ve výsledku mnohem dražší a časově náročnější, než by mohl být. Strohá specifikace zadání není zjednodušením práce, ale naopak.

Znám případ, kdy stál jeden měsíc projekťák firmu 230.000, vývojář 220.000 a tester 170.000.

Reálné platy ale měli tak poloviční - vzal se v potaz čas strávený vysvětlováním zadání, odchýlení se od deadlinu / odhadu a další data spojená s chybějícím zadáním. Rád vám ta zadání budu sepisovat za čtvrt milionu měsíčně... Vsadím se, že se vám i ve vašem případě vyplatím. Tak co, jak efektivně v týmu komunikujete?

Krok 2: Zefektivněte týmovou komunikaci

Jak na kvalitní zadání; co by mělo obsahovat?

- Přidejte deskriptivní titulek v jedné větě, který shrnuje daný úkol.
- Napište popis, který podrobně vysvětluje účel a kontext. Zapište, co přesně je potřeba udělat a proč je to důležité. Přiložte jakékoli relevantní background informace nebo reference na předchozí tasky, odkazy do dokumentace, kontakty a další.
- Definujte akceptační kritéria, která jasně specifikují, jak poznáme, že je úkol opravdu hotový. Kritéria by měla být jasná, stručná, nejlépe v bodech a hlavně by měla být měřitelná.
- Rozdělte ho do menších podúkolů a vzájemně je provažte, pokud je úkol komplexní.
- Definujte kritéria pro testování. Určete, jakým způsobem budete funkcionalitu testovat, nebo jakým způsobem by ji měl testovat tester. Popište v bodech krok za krokem, co má dělat. Specifikujte edge casy, validaci, možné chyby a další.
- Zaveďte tag, který bude zadání označovat jako neúplné. Na základě komentářů a tagu bude moct projekťák upravit specifikaci. To může sloužit i jako určitá metrika pro kvalitu vašich zadání.
- Zajistěte, aby se veškerá komunikace ohledně zadání odehrávala v komentářích zadání a nikoli v postranních kanálech jako je Slack, e-mail atd.
- Navštivte několik mítinků a vyhodnoťte jejich efektivitu. Zjistěte, co se tam řeší a jestli někdo dělá zápis nebo se mítink nahrává.

- Požadujte zrušení nebo zefektivnění mítinku (přidanou hodnotu - doplnění odhadů, vyjednávání o kompromisech…), pokud se tam vysvětluje to, co by mělo být v zadání, nebo se to ze zadání dokonce jen čte. Takový mítink je naprosto zbytečný.
- Podívejte se do nástroje pro správu úkolů a vyhodnoťte jednotlivá zadání.
- Požadujte, aby zaměstnanci udělali zápis nebo vložili screenshoty komunikace, pokud se komunikace ohledně zadání odehrává mimo nástroj pro správu úkolů.
- Dejte jasně najevo, kdo má za stav zadání odpovědnost, čí náplň práce to je.
- Zajistěte, aby měl projekťák dostatek času na sepisování zadání a zbytečně ho nezatěžujte dalšími činnostmi.

Kdo je přímo odpovědný za návrh API? Nikdo?

Máte návrh API a technickou? A mohl bych ji vidět? Firmy často nemají dokument, který by popisoval API jako celek. Dokument by měl obsahovat cíle, o kterých jsem psal výše, a jednotlivá architektonická rozhodnutí. Popis, proč jsme zvolili právě tuto technologii a ne jinou atd. Není třeba vypisovat všechno, ale aspoň ty nejdůležitější informace, na které by se mohl někdo ptát.

V praxi se stávají dvě situace. Firma nemá softwarového architekta nebo jinou osobu odpovědnou za návrh API a architektury. Pokud se tedy projekt nepovede, nikomu to na hlavu nepadá. Firma v takovém případě používá různé šablony a skeletony, které dává na všechny své projekty nebo všem svým zákazníkům a to bez rozmyslu. Prostě to tak dělají roky, je to osvědčený postup a mají to rychle. Stanovili jsme si nějaké cíle, takže je nám jasné, že každá aplikace může mít jiné a je potřeba ke každé aplikaci přistupovat individuálně.

Druhý problém je, že veškeré know-how drží v hlavě jeden zaměstnanec nebo skupina vyvolených. Co se stane v případě, že odejdou? Během výpovědní lhůty určitě dokumentaci dát dohromady nestihnou a na spoustu věcí zapomenou, protože jsou “přece jasné ne”? Až na to, že vůbec. Dokumentaci týkající se návrhu API a architektury udržujeme také proto, abychom mohli fungovat, i v případě, že nám někdo odejde a někdo jiný přijde.

Software sice není hmatatelný, jako například budova, ale pro náš byznys je stěžejní. Abychom mohli jednotlivé cíle naplnit, je potřeba všechno zdokumentovat. Zedník budovu bez návrhu taky nepostaví. Co se týče odpovědnosti, tak tu byste měli vyžadovat hlavně po svých dodavatelích nebo jako “motivaci” vašich zaměstnanců. Testeři mají často taky zodpovědnost za to, že funkcionalita funguje tak, jak má.

V případě, že budete projekt outsourcovat nebo najímat externí dodavatele, většina z nich po vás bude veškeré dokumentace vyžadovat - jak jinak se s projektem seznámí, že? Pokud je nemáte, buď s vámi nebudou spolupracovat, nebo to bude podstatně dražší, a nebo budou vyžadovat sepsání dokumentace v rámci spolupráce, tím pádem toho ve výsledku stihnete míň.

Krok 3: Budujte knowledge-base a určete odpovědnost

Jak vybudovat knowledge base (nejen) kolem API?

- Zvolte si vhodný nástroj pro správu knowledge, ve kterém se dá snadno vyhledávat. Dobrou volbou může být Notion nebo samostatný knowledge base repozitář, případně adresář s markdown soubory.
- Definujte strukturu knowledge base a členění informací. Vhodnou adresářovou strukturu můžete vytvořit pomocí PARA metodologie.
- Vytvořte step-by-step dokument popisující projekt a API jako takové. Popište, jak API spustit, nainstalovat, zda jsou potřeba externí knihovny, import databáze atd.
- Přiložte k dokumentaci zápisky ze schůzek, návrhy architektury, threat model a další relevantní údaje, jako je odkaz do správce credentials atd.
- Určete, kdo je odpovědný za návrh API a architektury, a definujte, co tato odpovědnost obnáší.

Ušetřete čas a peníze, zapřáhněte robota!

Do teď jsme řešili hlavně procesní záležitosti, které se těžko definují. Teď se posuneme k něčemu praktickému. Více než 90 % aplikací je tvořeno kódem a knihovnami třetích stran. Maximálně 10 % je kód, který napsali vaši vývojáři, zbytek tvoří Node.js, Spring atd. Tyto projektové závislosti jsou neustále aktualizovány, protože, stejně jako váš projekt, se stále vyvíjí a vylepšují. Také ale mohou obsahovat bezpečnostní zranitelnosti. Každá závislost do vaší aplikace přináší nové potenciální hrozby a vy musíte sledovat, zda se kvůli nim vaše API nestává zranitelným.

Můžete si myslet, že je to v pohodě, protože vaše API má jen 10 závislostí. Ve výsledku jsou jich ale tisíce, protože každá z těch závislostí má další závislosti, ty mají další závislosti atd. Pokud byste měli pravidelně každou z těchto závislostí sledovat, nedělali by vaši zaměstnanci nic jiného. Proces aktualizace a monitoringu závislostí se dá automatizovat pomocí nástrojů jako je Dependabot, Renovate a další.

Tyto roboti pravidelně prochází vaše závislosti a kontrolují, zda neexistují aktualizace (nikdo nechce pracovat se zastaralým systémem). Pokud aktualizace existují, automaticky je upgradují nebo vás na ně upozorní. Zároveň ale hlídají, jestli některá ze závislostí neobsahuje nějakou známou zranitelnost. Pokud je některá závislost zranitelná, pokusí se ji upgradovat, existuje-li verze s bezpečnostní záplatou, a nebo ji downgraduje v případě, že je to pro vás opravdu zásadní problém.

Většina robotů je konfigurovatelná a určitě vám nebudou dělat žádné změny v závislostech, dokud jim to explicitně neřeknete. Je to velmi užitečný pomocník, který pomůže udržet vaše API aktuální a předcházet různým typům zranitelností. Jejich konfigurace nezabere víc než pár minut, všechno se jednoduše nakliká a pak už z toho jen profitujete po celou dobu životnosti vašeho projektu. Konfigurace je většinou přenositelná, takže ji můžete snadno aplikovat i na další projekty.

Doporučuju nastavit automatický běh robota, například každé úterý v 6 ráno. Robot automaticky připraví report a pull requesty, které na začátku pracovní doby během pár minut zprocesujete. Nastavte robota, aby běžel pokaždé, když někdo vytvoří pull request se změněným souborem se závislostmi (package.json, yarn.lock atd.), aby se zkontrolovala verze přidaných závislostí a jejich bezpečnost.

Roboti umí dodržovat tzv. semver verzování, takže pokud ho používáte, nemělo by se stát, že vám nainstaluje něco, kde budou breaking changes (pokud pomineme, že někdy za to může vývojář dané knihovny a nikoli robot).

Krok 4: Automatizujte správu závislostí

Předcházejte zranitelnostem a zajistěte best practices pravidelně

Existuje nástroj, který automaticky odhalí bezpečnostní rizika, udělá automatické code review a je snadné ho integrovat do workflow při vývoji API. Jde o statickou analýzu kódu. Úkolem statické analýzy kódu je podívat se na kód aplikace a podle známých patternů odhalit bezpečnostní rizika jako je SQL injection, SSRF a další. Statická analýza kódu ale taky rozumí samotnému programovacímu jazyku a dokáže identifikovat kusy kódu, kde je možné něco zlepšit. Umí optimalizovat algoritmy nebo funkce podle definovaných pravidel, zajistit jednotné formátování a další.

S její pomocí nejenže odhalíte bezpečnostní zranitelnosti, ale taky naučíte vaše programátory lépe programovat. Zejména v raných fázích projektu jde o velmi cenného a šikovného pomocníka, který bude vaše vývojáře vést k tomu, aby používali osvědčené best practices a psali čistý a optimalizovaný kód. Je ale vhodná po dobu celého životního cyklu aplikace.

Statická analýza kódu identifikuje přesný řádek kódu, kde se daný problém nachází, což vývojářům usnadňuje spoustu práce. Dokáže popsat daný problém, poradit s jeho odstraněním a v některých případech sama navrhne úpravu nebo kód přepíše.

Nevýhodou však je, že s jejím použitím vznikne relativně dost tzv. false positives, tedy chyb, které vlastně nejsou chyby.

Statická analýza, stejně jako jiné automatické nástroje, nezná kontext vaší aplikace a nedokáže jednoznačně posoudit, jestli je daný problém relevantní nebo ne. Snadno se stane, že nahlásí SQL injection zranitelnost, ale to, že jí předcházíte podmínkou na předchozím řádku, už nevidí.

I přes tuto nevýhodu ji však doporučuju. Samotné nastavení je velmi jednoduché a postupem času se vyřeší i zmíněné false positives. Prakticky všude je možné nastavit, že danou chybu nechcete zařadit do hlášení atd.

Statickou analýzu není tak snadné si vybrat. Nejspíš budete dělat kompromis. Vždy ale závisí na vašem prostředí a technologickém stacku. Máte dvě možnosti. Statická analýza, která zná daný jazyk do hloubky, a statická analýza, která podporuje velké množství programovacích jazyků (tzv. do šířky). Pokud máte celé API v jednom programovacím jazyku a neplánujete přidávat další, zvolte takový nástroj, který zná daný jazyk do hloubky. Pokud máte různorodý stack, použijte takový nástroj, který vám pomůže zakomponovat co nejvíc programovacích jazyků.

Pokud má statická analýza hlubokou znalost daného jazyka, bude přesnější a pomůže řešit i komplikovanější situace. Pokud jdete do šířky, přicházíte o hlubokou znalost a budete se potýkat s jistými omezeními. Ať už zvolíte cokoli, určitě neuděláte chybu.

Mezi známé statické analýzy kódu patří ESLint nebo CodeQL. Určitě jich najdete víc v závislosti na tom, jaký programovací jazyk používáte.

Krok 5: Automatizujte pomocí vhodných security nástrojů

Jak si vybrat statickou analýzu kódu správně?

- Poznejte své prostředí - programovací jazyky, frameworky a platformy, které používáte.
- Definujte, co od analýzy čekáte na základě vašich cílů (optimalizace, bezpečnost, výkon…).
- Stanovte, zda použijete statickou analýzu do hloubky, nebo do šířky, v závislosti na vašich očekávání a prostředí.
- Udělejte pečlivý research nalezených nástrojů. Zaměřte se na to, co statická analýza pokrývá (bezpečnostní zranitelnosti, code smell, best practices, optimalizace výkonu…).
- Podívejte se, zda poskytuje možnost konfigurace a customizace a jak je tato možnost flexibilní.
- Ověřte, zda půjde nakonfigurovat takovým způsobem, aby se adaptovala na váš současný kód a standardy.
- Zkontrolujte, zda bude snadné zapojit ji do CI/CD, IDE a dalších prostředí.
- Vyzkoušejte si trial a zaměřte se na false positives, zda je efektivně filtruje nebo ne a zda je můžete filtrovat ručně.
- Podívejte se, jak vypadají reportované nálezy, zda jsou dobře popsané. Nabízí nástroj grafy a jiné vizualizace a dají se customizovat?
- Zaměřte se na to, jaký bude dopad na vaše workflow - je nástroj efektivní z hlediska výkonu?
- Podívejte se, zda se vám líbí uživatelské rozhraní a bude se vám s ním dobře a jednoduše pracovat.

- Prověřte, jak silná je uživatelská základna nebo komunita daného nástroje, zda existuje nějaké fórum, FAQ, dokumentace a jakým způsobem můžete kontaktovat podporu.
- Porovnejte cenu jednotlivých nástrojů.
- Přečtěte si podmínky používání a dokumenty související s bezpečností a privacy, aby se nestalo, že se vaše reporty nebo kódy dostanou tam, kam by neměly.
- Vyzkoušejte si zvolený nástroj na menším projektu nebo mikroslužbě - udělejte PoC.
- Nakonfigurujte lokální běh statické analýzy kódu.
    - Nakonfigurujte přeformátování kódu po uložení souboru v IDE.
    - Nainstalujte pre-commit hook, který spustí analýzu před vytvořením git commitu a zkontroluje kód před tím, než se dostane do repozitáře.
        - Použijte například husky hook.

Jsou vaše credentials v bezpečí? Pochybuju...

Jednou jsme ve firmě pořádali hackathon, jehož cílem bylo seznámit účastníky s technologií Node.js a Google Cloud Platform. Účastníci měli vytvořit jednoduchou aplikaci, nahrát ji do cloudu a potom s ní interagovat. Pokud chcete aplikaci nahrát na Google cloud, musíte se nějak autentizovat a autorizovat vůči Googlu. K tomu slouží tzv. service account, což je forma privátního klíče, který Google Cloud používá. Pro účastníky jsme připravili šablonu a infrastrukturu, včetně service accountu a nahráli jsme to na GitHub do veřejného repozitáře, aby k tomu měli jednoduchý přístup. Hackathon dopadl dobře, byli jsme nadmíru spokojení. To se ale nedá říct o momentu, kdy jsme od Googlu obdrželi fakturu za využití cloudových služeb za 20.000 dolarů. Zjistili jsme, že někdo pomocí automatického nástroje pro hledání credentials a secrets našel náš service account na GitHubu. Stáhl si ho a vzhledem k vysokým oprávněním service accountu si vytvořil virtuální stroj na naší infrastruktuře a začal tam těžit kryptoměnu Monero.

S podobným problémem, kdy se credentials - hesla, tokeny, klíče, atd. vyskytují na místech, kde nemají co dělat, se setkávám snad dennodenně. Hesla lítají po Slacku, v e-mailech, vyskytují se v dokumentacích na Google Drivu, kam má přístup polovina firmy, jsou v privátních nebo veřejných repozitářích atd. A to nemluvím o zbytečně velkých oprávněních, které tyto credentials často mají.

Otázkou tedy je, kde tyto údaje ukládat a jak je efektivně spravovat. Ne každý by totiž měl mít přístup ke všem credentials. Možností je hned několik. Můžete použít některé správce hesel, například 1Password má různé integrace a dá se tam celkem dobře spravovat přístup a další věci. Není to však ideální. Osobně mám rád Vault od společnosti HashiCorp, což je open source pro správu credentials. Výhodou jednoznačně je, že si ho můžete hostovat sami a s nikým tak své credentials nesdílíte. Samozřejmě můžete použít i AWS secrets manager a další nástroje.

Asi nemusím mluvit o citlivosti těchto credentials, to se omílá stále dokola. Dopad ale může být opravdu velký, jako tomu bylo například v mém případě. Za většinu služeb, ke kterým přistupujeme pomocí credentials, platíme v tzv. pay as you go módu. Pokud credentials uniknou a někdo je zneužije, dozvíme se to většinou až s přijatou fakturou. Doporučuju tedy navíc zapnout billing notifikace nebo nastavit limit pro dané služby všude, kde to jen jde.

Často se stává, že bývalí zaměstnanci vynáší tato data nebo na firemní infrastruktuře rozbíhají vlastní projekty. Bohužel to vůbec není ojedinělé. Držte se tedy principu least privilege a dejte svým zaměstnancům jen nezbytně nutná oprávnění pro vykonávání jejich práce. Tomu bude možná potřeba trochu přizpůsobit procesy - např. citlivější operace bude dělat tým lídr atd.

Credentials mimo systém pro jejich správu je velmi obtížné monitorovat. Existují sice nástroje, které se o to snaží, ale jejich úspěšnost je velký otazník.

Některé credentials mají jasné a známé patterny, ale v případě tokenů a hesel hledáme jehlu v kupce sena. Jde hlavně o nějakou firemní politiku, procesy a nastavení firemní kultury, kterou budou zaměstnanci respektovat.

Krok 6: Pořiďte si nástroj na správu credentials

Co by měl nástroj pro správu credentials nabízet?

- šifrování a bezpečné / zabezpečené úložiště
- centralizovanou správu credentials, která zmírňuje riziko úniku dat
- granulární přístupová oprávnění na základě atributů nebo uživatelských rolí
- jednoduchou rotaci a změnu credentials
- logování, auditování a případný monitoring
- integraci s CI/CD nebo API včetně kvalitní dokumentace
- prostředí pro jednoduché sdílení credentials v rámci týmu nebo firmy
- bezpečnostní certifikace typu SOC2, open source nástroj

Nepište dokumentaci, generujte ji!

Je známo, že vývojáři neradi píšou dokumentace. A nejen vývojáři, spousta technických i netechnických pracovníků to má stejně. Některé typy dokumentací, jako je například technická nebo architektonická, je opravdu potřeba napsat. Nicméně API dokumentaci psát nemusíte! Jednoduše ji vygenerujete.

Jako vývojář vyloženě nesnáším jakákoli PDFka, Word dokumenty a jiné rádoby “user friendly” dokumentace. Jsou k ničemu, nepoužitelné. Často není možné cokoli jednoduše zkopírovat, blbě se v tom hledá, nemůžu poslat odkaz na konkrétní metodu, ale hlavně s tím nemůžu nijak interagovat. Taková dokumentace není interaktivní a vypovídá zejména o tom, že pracuju s něčím nejspíš z doby kamenné.

Od API dokumentace očekávám, že bude interaktivní a snadno přístupná. Bude obsahovat nejčastější příklady použití, datový model, příklady odpovědí včetně chyb. Skvělou dokumentaci má například Stripe. Když vývojář dělá nějaký research nebo začíná pracovat s dokumentací, rád si dané funkcionality zkouší. Rád danou API funkci jen tak zavolá bez jakýchkoli parametrů, aby viděl aspoň nějakou odezvu. Udělejte to samé s PDFkem – strávíte dvacet minut nastavováním a pak zjistíte, že API nefunguje… Uživatelská zkušenost je tragická a je jedno, jestli jde o interní nebo veřejnou dokumentaci.

Pokud firma nemá kvalitní API dokumentaci ke svému produktu, svým klientům jejich produkt nedoporučuju. Jednoduše proto, že bude obtížné cokoliv o daných funkcích zjišťovat. Této problematice jsem se věnoval s Adamem Bártou v našem rozhovoru o tom, na co si dát pozor při výběru informačního systému. Dokumentace šetří čas a peníze vašim klientům, zákazníkům nebo vývojářům a členům týmu.

Existují standardizované formáty jako je OpenAPI (open source), pomocí kterých vytvoříte krásnou, přehlednou a interaktivní API dokumentaci. Vzhledem k tomu, že je standardizovaná, tak s ní umí pracovat nejen lidi, ale i různé nástroje. Díky dokumentaci v tomto formátu například budete moct automaticky testovat vaše API na známé zranitelnosti atd.

Různé programovací jazyky, jako je Java nebo Node.js (Javascript) mají pluginy, které umožňují dokumentaci automaticky generovat. Vývojář jednoduše nainstaluje OpenAPI závislost, jednotlivé funkce pak vhodně otaguje a pomocí skriptu vygeneruje krásnou dokumentaci. Díky tomu je možné nastavit automatickou kontrolu, validaci příchozích a odchozích dat atd. To znamená, že vám tento plugin taky pomůže zajistit větší bezpečnost vašeho API.

Pravdou je, že nástroj nejspíš nezná kontext vaší aplikace, takže vývojář přece jenom bude muset v pár větách popsat, co daná funkce dělá, aby se vygeneroval relevantní popis. Vzhledem k celkovému zjednodušení a přínosu této automatizace je to ale zanedbatelné. Hlavně vývojář ví, co jaká funkcionalita dělá nebo co ji spouští, takže by neměl být problém ji popsat. Pokud ano, může za to ve většině případů prostá lenost.

Když firmám pomáhám s psaním dokumentace, držím se následujících bodů:

- Nejdřív otaguju všechny endpointy, abych vytvořil celkovou strukturu API.
- Potom endpointy seskupím, aby byly pohromadě.
- Každé skupině se pak věnuju zvlášť a dám dohromady popis datového modelu pro vstup a výstup včetně chybových hlášení.
- V poslední řadě se věnuju HTTP hlavičkám, parametrům a dalším věcem, jako je například autentizace. Díky tomu získám přehled o celém API a můžu popisovat jednotlivé funkcionality, pokud jsem to nedělal průběžně (ne vždy je všechno jasné).
- Na závěr všechno zoptimalizuju a upravím tak, aby se ve struktuře nevyskytovaly duplicity atd. Jakmile je práce hotová, nastavím automatizace a je to.

Takhle vypadá proces, pokud přijdu k aplikaci, která nemá dokumentaci vůbec. Pokud jde o nový projekt, vše se taguje podle toho, jak plyne vývoj.

Dokumentace se pak automaticky generuje v tzv. pipeline. To je proces, kdy se aplikace sestaví, spustí se testy a nahraje se na server. Jedním z kroků často bývá právě vygenerování aktuální dokumentace a její nahrání na příslušné místo. Jako vývojář tedy prakticky nehnu prstem. Jediné, co dělám, je, že taguju funkcionalitu a sem tam něco slovy popíšu. Automatizaci nakonfiguruju jednou, je přenositelná, takže ji můžu použít i na dalších projektech.

Takto vygenerovaná API dokumentace by měla být základem každého API. Díky ní máte o vašem API přehled - vytvořili jste inventář. Ten vám bude do budoucna sloužit i v situacích jako je incident response, monitoring a další.

Na základě takové dokumentace je možné říct, zda je API optimalizované nebo z ní můžete vygenerovat use casy, které zjednoduší práci lidem, kteří vaše API budou používat.

Krok 7: Pište dokumentaci a budujte inventory

Jak by měla vypadat dokumentace?

- Obsahuje úvod a popisuje projekt nebo API jako takové. Vysvětluje, s čím vlastně dotyčný pracuje a vytváří kontext.
- Definuje první kroky, které se musí podniknout, aby se mohlo API začít používat. Popisuje krok za krokem tento proces.
- Obsahuje code snippets v různých programovacích jazycích, pokud je to nutné.
- Detailně popisuje jednotlivé endpointy. Obsahuje metody (GET, POST…), popis funkcionality, příklad použití, specifikuje datový formát (JSON…), požadavky na přihlášení.
- Obsahuje příklad requestu a response spolu s detailním popisem jejich struktury nebo datového modelu.
- Definuje veškeré parametry, datové typy, validační pravidla, error zprávy, response kódy a jejich význam.
- Popisuje příklady použití nebo nabízí tutoriál včetně interaktivních prvků.
- Obsahuje troubleshooting a FAQ.
- Definuje verzování API a poskytuje changelog.
- Umožňuje v dokumentaci snadno vyhledávat a navigovat.
- Popisuje, jakým způsobem se API udržuje a spravuje (např. v jaké dny mohou hrozit výpadky atd.).

Co vám o code review neřekli a proč ho děláte špatně

Drtivá většina firem, ve kterých jsem pracoval, dělá code review špatně. Jak to vypadá? Zaměstnanec kontroluje, zda je dodrženo správné formátování, zda je název funkce deskriptivní, jestli je kód obecně dobře čitelný nebo jestli jsou dodrženy některé best practices typu návrhové vzory atp. “To je ale dobře…”, řeknete si. Ano, je to dobře, ale tohle přesně zvládnou automatizované nástroje, jako je statická analýza kódu. Nedává žádný smysl, natož finanční, aby tuto činnost dělal člověk.

Pokud kontrolujete práci například po své sekretářce (to je taky takové code review), na co se soustředíte? Na gramatiku? Tu ve většině případech opraví Google dokument. Na styl písma? Ale no tak… Soustředíte se na to, jestli vůbec udělala to, co jste po ní chtěli. Porovnáváte zadání oproti práci samotné a jejímu výstupu.

Abych dokázal udělat kvalitní code review, musím znát kontext daného projektu. Musím do něj být zasvěcený a musím znát jeho vizi a cíle. Bez toho nedokážu určit, jestli je daný kód optimálně napsaný. Pokud je váš cíl mít rychlé API, tak vás nezajímá, jestli daná funkce funguje, zajímá vás, jak je rychlá a efektivní.

K tomu, abych se dokázal vžít do svého kolegy, kterému dělám code review, potřebuju kvalitní zadání. Musím vědět, co je úkolem a jak ověřím, že je úkol splněný. Přes to nejede vlak, budeme jen vařit z vody, pokud tohle nemáme.

Na základě toho můžu krok za krokem porovnávat naprogramovanou funkcionalitu oproti zadání. Hledám slabá místa, která by šla optimalizovat nebo udělat lépe (v závislosti na cíli). Můžu porovnávat čitelnost, jmennou konvenci, ale soustředím se hlavně na funkcionalitu.

Příslušný kód bych si měl stáhnout a spustit si ho lokálně. Vyzkoušet, jestli dělá opravdu to, co má. Znáte přece ten problém typu: “Ale na mém počítači to funguje…”? Tak něco takového taky ověřujeme, jestli to funguje i někde jinde a za jakých podmínek. Není nutné to dělat s každou funkcí, ale některé funkcionality jsou tak složité, že vývojář příslušné chování prostě nevykouká a musí si program zapnout.

Jako člověk, který dělá code review, se tedy soustředím na věci, které souvisí s kontextem. Kontext je něco, co za nás žádný automatický nástroj nevyřeší. Zatím stále žijeme v době, kdy kontext pochopí jen člověk. Záležitosti týkající se kontextu mohou být různé optimalizace, bezpečnost, UX nebo celkový návrh, použitelnost, správnost oproti zadání, dodržení metrik, dostatečné logování, databázová struktura, efektivita algoritmu a další.

Code review je náročný proces a měli byste počítat s tím, že bude trvat cca 20 % času, který jste strávili na vývoji dané funkcionality. Tento čas je potřeba přičíst k odhadu včetně nějakého bufferu, protože může dojít k výměně názorů a opravám.

Dalším problémem při code review je komunikace. Vývojář, který dělá code review, musí být spíš takový mentor, ne pruďas.

Setkal jsem se zprávami typu: “Co to je tohle?”, “Už jednou jsme si říkali, že takhle ne…”, “WTF” nebo “Udělej to jinak”. Takový člověk je naprosto nekompetentní k jakékoli vedoucí pozici nebo k činnosti code review. Co se z takových zpráv dozvíte nebo naučíte kromě toho, že je váš kolega blbec?

Code review je diskuze. Jako vývojář bych se měl například ptát, proč je to zrovna takhle, třeba mi něco uniklo. Měl bych se ptát na názor protistrany: “Toto by mohlo být lepší řešení, protože…, co si o tom myslíš?”. Rozhodně bych měl vysvětlit, proč si myslím, že je něco špatně nebo by se mělo zlepšit, abych buď mentoroval nebo aby mi bylo vysvětleno, že jsem mimo. Z code review se učíme všichni, ne jeden. Neměl bych si připadat jako nadřazený jen proto, že dělám code review. Často dělají code review i junioři, protože je to jedna z nejlepších cest, jak se naučit spoustu nových věcí.

Často jsem se setkal s tím, že byl úryvek z code review zveřejněn a sdílen s celou firmou, aby se tam ukázalo, jaké daný jedinec dělá chyby. Nedělejte to, nebo to aspoň anonymizujte. Vaši zaměstnanci pak tento proces nebudou mít rádi a bude to jeden z hlavních důvodů, proč opustí vaši firmu. Budujte no-blame kulturu.

Spousta seniorů opouští firmy proto, že mají pocit, že v dané firmě dosáhli vrcholu. Nikdo je nechallenguje. Sám jsem takto z několika firem odešel. Code review je výborný nástroj, jak zdravě challengovat své zaměstnance mezi sebou. Dává jim to obrovský prostor k osobnímu růstu, tak toho využijte a naučte se dělat ho správně.

Krok 8: Naučte se dělat code review správně

Co zkontrolovat během code review od A do Z?

- Je kód snadno čitelný a srozumitelný?
- Je jmenná konvence konzistentní a dává smysl?
- Je kód správně naformátovaný (odsazení, délka řádku…)?
- Je kód správně logicky organizovaný do funkcí, tříd a modulů?
- Obsahuje kód komentáře s vysvětlením složitějších konstrukcí, komplexních funkcionalit nebo algoritmů?
- Neobsahuje kód code smells nebo anti patterny?
- Splňuje funkcionalita všechny požadavky zadání?
- Jsou v kódu nějaké logické chyby, které by měly být prodiskutovány?
- Počítá funkcionalita s edge casy nebo jinými scénáři?
- Obsahuje funkcionalita dostatečný error handling?
- Je v kódu jakýkoli prostor pro zlepšení a optimalizaci výkonu?
- Je kód modulární a znovupoužitelný?
- Existuje možnost extrakce společné logiky do separátních modulů / knihoven?
- Může být funkcionalita snadno rozšířena nebo modifikována bez provedení výrazných změn?
- Jsou chybová hlášení správně odchycena a zalogována?
- Jsou chybové zprávy deskriptivní, uživatelsky přívětivé a zároveň dostatečně obecné, aby nevyzradily cenné informace pro útočníky?
- Jsou v kódu nějaké bezpečnostní zranitelnosti?
- Je uživatelský vstup validován a sanitizován?

- Jsou citlivá data bezpečně zpracována, šifrována a ukládána?
- Obsahuje funkcionalita dostatečné množství unit nebo integration testů?
- Validují testy dostatečně očekávané chování?
- Nechybí nějaké testovací scénáře, edge casy atd.?
- Je kód adekvátně zdokumentovaný?
- Obsahuje funkcionalita podrobný návod, jak ji zprovoznit, nakonfigurovat, spustit, atd.?
- Odpovídá stav dokumentace stavu kódu? Neliší se? Není jedno oproti druhému zastaralé?
- Dodržuje kód zavedené standardy a best practices v oblasti vývoje?
- Neporušuje kód firemní nebo jiné standardy / guide lines?
- Jsou zde nějaké konflikty s větví, do které se bude funkcionalita mergovat?
- Jsou commit zprávy deskriptivní a dodržují stanovené standardy a guide lines?
- Je váš feedback konstruktivní a obsahuje otázky nebo návrhy pro vylepšení funkcionality?

Testujte, testujte, testujte... bez vynaložení lidských zdrojů samozřejmě

Automatické testování jako je unit, integration testing atp. je pro vás velmi výhodné. Tady už vývojáři sice programují samotné testy, ale profitujete z nich po celou dobu životnosti aplikace. Konfigurace testů je přenositelná a je možné ji aplikovat na všech projektech.

Poté (nebo předtím), co vývojář naprogramuje nějakou funkcionalitu, naprogramuje testy, které ověří testovací scénáře popsané v zadání. Z mého pohledu nejde ani tak o programování, jako o volání příslušných funkcí a porovnávání výstupů. Tyto testy pak automaticky spouštíte kdykoli to potřebujete. Nespornou výhodou je, že tyto testy jsou jednoznačné. Tester může pokaždé zvolit jiný postup, ale program je program; stále dělá to samé a na nic nezapomene.

Často se stane, že měníte nějakou funkcionalitu, ale nechcete ovlivnit výstup - API musí vracet stále to samé. Pokud nemáte automatické testy, musí přijít QA tester, který to ručně ověří. Takhle to bude pokaždé, kdykoli něco upravíte. Už asi chápete, kam tím mířím. Je velmi snadné pouštět testy stále dokola a jednoznačně ověřovat funkcionalitu. Šetří to zdroje nejen testerů, ale i lidí, kteří dělají code review.

V předchozí kapitole jsem psal, že některé funkce není lehké vykoukat a vývojář si musí kód stáhnout a spustit si ho lokálně, aby pochopil, co se tam vlastně děje a jak to funguje. Pokud máte automatické testy, stačí když se podívá na ně. Hned si ověří flow a snadno zjistí, zda funkcionalita funguje. Díky tomu ho ale taky může napadnout scénář, který funkcionalitu rozbije a může ho vyzkoušet.

Testy sice zabírají nějaký čas, nejspíš jde o nejnáročnější úkol, který v tomto e-booku popisuju, ale přínosy rozhodně nejsou zanedbatelné. Na automatických testech snadno dokážete demonstrovat funkčnost i svým zákazníkům nebo byznys ownerům, můžete generovat různé reporty a nastavit velké množství metrik. Bohužel jde o činnost číslo dva, kterou vývojáři neradi dělají. Ale i s tím se dá pracovat. Například tak, že tuto činnost outsorcujete nebo najmete někoho, kdo bude testy psát. Většina QA inženýrů v dnešní době píše právě automatické testy v Cypressu a dalších nástrojích.

Testy by měl psát nejlépe vývojář, který programuje danou funkcionalitu. On je přece tím, kdo má do problematiky největší vhled. Pokud je bude psát někdo jiný, je potřeba, aby rozuměl kontextu aplikace a API, znal cíle a zadání bylo kvalitně popsané. Bez toho není možné napsat kvalitní testy.

Dbejte na to, aby i testovací scénáře prošly procesem code review. Může se stát, že zapomenete ověřit nějaký významný detail nebo budete chtít pokrýt víc scénářů. Každá funkcionalita by měla mít alespoň jeden test – tím je happy path scénář. Jde o test, který ověří funkčnost za ideálních podmínek.

To znamená, že uživatel bude postupovat podle popisu, zadá validní data, zavolá endpointy ve správném pořadí atd. Z happy path scénáře se pak dají dělat různé odnože, které budou testovat efektivitu validace, výkon, rychlost, neočekávaný průběh volání atd.

Často od klientů dostávám otázku: “Co máme testovat? Těch funkcionalit je tolik, že na testování všeho nemáme budget ani kapacity.” Testujte to, co je pro vás důležité. Podívejte se na své cíle – ty by vás měly inspirovat a udat vám směr. Testujte kritické části aplikace. Může jít o validaci vstupních dat, autorizační logiku (ta by měla být pokryta celá, tedy 100% test coverage). Zaměřte se na byznys flow - jaká byznys funkce vaší aplikace se rozhodně nesmí rozbít? Je to přihlašování, objednávka, tvorba faktur? Napište na to testy. Určitě bych testoval logiku, kde dochází k transakcím typu převod peněz, platba, synchronizace dat, exporty, atd.

Krok 9: Automaticky testujte své API

Co nezapomenout otestovat?

- API endpoint – end-to-end test, který prověří status kódy a vrácená data. Ověří parametry, metody a správnost chování endpointu.
- Validaci vstupu – zpracování validních a nevalidních dat a ověření chování v případě edge casů.
- Error handling – reakce API na výjimečné a chybové situace.
- Autentizace a autorizace – kontrola access control mechanismu a přihlašovacího flow, včetně kontroly expirovaných tokenů atd. Ověřte přístupová oprávnění v závislosti na uživatelských rolích a dalších atributech.
- Výkon – doba odezvy, propustnost, škálovatelnost, zvládání velkého počtu požadavků, kontrola možných memory leaků.
- Integrační testování – testování celé funkcionality včetně zápisu do databáze a použití dalších služeb. Simulace uživatelského flow.
- Bezpečnost – identifikace potenciálních zranitelností (SQL injection, XSS, selhání autorizace…).
- Verzování – testujte různé verze svého API a zjistěte, jak reaguje na určité scénáře a zda se všechny verze chovají tak, jak se od nich očekává.
- Resilience – jak se API chová v případě výpadku služby nebo ztráty spojení? Testujte, zda se služba sama vzpamatuje, bezpečně se ukončí, atp.
- Developer experience – jak snadné je s API pracovat? Kolik to vyžaduje úsilí a provolaných endpointů?

Víte, co se s vaším API právě teď děje?

Víte, kam teď kliknout, abyste mi dokázali říct, zda vaše API plní stanovené cíle, zda dochází k útokům nebo jejich předcházení, zda všechny služby běží tak, jak se očekává, nebo prostě jen že aplikace běží a funguje? K tomu slouží monitoring. Pod monitoringem si často představujeme monitorování výkonu, zdrojů nebo počtu uživatelů. Co ale bezpečnostní monitoring? Nepokoušel se teď někdo 100x za sebou přihlásit? Kdo a jak často se pohybuje a přihlašuje do administrace?

Možná si říkáte, že tyto informace nejsou důležité, ale v některých případech dokonce porušujete zákon, pokud takové informace neznáte. Zabývají se jimi různé regulace a dokonce vyžadují archivaci těchto údajů. Jak je tyto data ale získáme?

K tomu slouží logování. Vývojáři často API logují za účelem debuggingu, aby věděli, jaký je průchod aplikací a co se tam děje. Na základě toho dokážou určit, co se případně pokazilo a jak to napravit. To je dobrý start. Z logů by měli být schopni poznat, kdo, kdy, co, jak a proč dělal. To znamená, že logují uživatele, který danou akci vykonával, kdy k tomu došlo, o jakou šlo funkci nebo endpoint, jaká data uživatel poslal nebo jaké byly vstupní parametry. Díky tomu je možné uživatelský postup replikovat.

Často se ale logují věci, které by se logovat neměly. Vývojáři běžně logují celý request a nefiltrují citlivé údaje. Stává se tedy, že se zaloguje access token, zadané heslo atd. Dokonce jsem u jednoho klienta řešil, že podpora aplikace funguje tak, že se vývojář podívá do logu, odkud si stáhne přihlašovací údaje daného uživatele. Přihlásí se s nimi do aplikace a v jeho účtu řeší, co se tam asi tak stalo, že mu aplikace nefunguje správně. Například projekt Slope (Solana kryptoměnová peněženka) logoval privátní klíče (seedy) peněženek do nástroje Sentry. V tomto nástroji si logy někdo zobrazil a vykradl krypto aktiva v hodnotě 5 milionů dolarů.

Nelogujte citlivé údaje a hlavně nelogujte celé objekty. Vždy pečlivě ověřte a vyfiltrujte všechno, co by v logu být nemělo (hesla, tokeny, uživatelské údaje, session ID, atd.). Pokud potřebujete zalogovat referenci na uživatele nebo session ID, zahashujte ho a uložte hash. Dejte si pozor, jestli nelogujete faktury, firemní know-how, duševní vlastnictví, kreditní karty, zdravotní záznamy a další data, jejichž únik by mohl poškodit nejen vás, ale i vaše uživatele.

Jaké funkce byste měli logovat a monitorovat? Přistupoval bych k tomu stejně jako v případě testů. Určitě bych logoval high value transakce (převody peněz), transakce důležité pro byznys use case (registrace, tvorba záznamů), citlivé operace (změna hesla, reset hesla, změna e-mailu), validační selhání (špatně zadaná vstupní data), autorizační selhání (uživatel chtěl zobrazit něco, na co nemá oprávnění).
Pokud budete logovat tyto informace, můžete je pomocí tagů seskupit a následně pro každý z tagů vytvoříte monitorovací pravidlo v nástrojích typu Grafana.

Monitorovací systém sbírá logy podle vámi stanovených pravidel a na základě metrik je nějakým způsobem vyhodnocuje a zobrazuje. Monitorovací systém je možné napojit i na systémy typu PagerDuty. Tento nástroj nás dokáže notifikovat například pomocí SMS nebo voláním, že se děje něco divného a měli bychom to jít řešit - například nastal nějaký bezpečnostní incident, aplikace neodpovídá, výrazně se snížil výkon atd.

Pro zajištění bezpečnosti a kvality je logování včetně monitoringu nezbytné. Nestačí však jen logovat a monitorovat. Někdo ty logy a monitoring taky musí sledovat. Často to dělá DevOps oddělení nebo admini, ale je prakticky na vás, komu to přiřadíte. Dívejte se na monitoring průběžně jednou denně, například ráno. Každý týden, měsíc nebo kvartál vypracujte report. Pokud máte stanovené cíle, monitoring vám pomůže s jejich kontrolou, je pro vás tedy velmi důležitý. Report může sloužit jako podklad s informacemi o naplnění cílů pro vaše nadřízené nebo kolegy.

Krok 10: Logujte a monitorujte

Jaké jsou best practices pro logování API?

- Logujte data ve strukturovaném formátu jako je JSON. Tím se zjednoduší analýza, vyhledávání a propojení s monitorovacími systémy.
- Použijte konzistentní formát pro všechny logy, aby bylo snadné jim porozumět, analyzovat je a vyhledávat v nich.
- Logujte důležité informace, jako jsou požadavky, odpovědi, parametry a chyby.
- Nelogujte jakékoli citlivé údaje.
- Použijte logovací úrovně jako je DEBUG, INFO, WARNING, ERROR pro různé události.
- Povolte konfiguraci logovacích úrovní, která umožňuje aktivaci a deaktivaci logování na různých prostředích.
- Logujte relevantní kontext, jako jsou uživatelské ID, verze API, metadata, IP adresy a další, které vám pomohou s analýzou a identifikací problémů.
- Logujte dostatečné informace o chybách, jako jsou zprávy a kódy chyb.
- Sdílejte logovací záznamy jen s těmi členy týmu, se kterými je to nezbytně nutné, a udržujte je v bezpečném úložišti.
- Použijte osvědčené logovací nástroje a knihovny, nevytvářejte si vlastní.
- Monitorujte a analyzujte jednotlivé logy, sledujte provoz a identifikujte potenciální problémy.
- Použijte nástroje pro centralizovaný sběr a analýzu logů.
- Zdokumentujte, jak správně logovat, a uveďte několik příkladů správných logů.

Závěr

Aplikační bezpečnost rozhodně není nový obor, ale do povědomí se začíná dostávat hlavně v poslední době. API se totiž stávají novým vektorem útoku a vstupní branou pro útočníky. Bohužel spousta CTO, head of development nebo manažerů vývojových týmů ji chápe špatně. Myslí si, že aplikační bezpečnost je zajištěna tím, že je kód nasazený v cloudu, firma používá WAF nebo VPN apod. Na to, že by jejich vývojář mohl napsat nebezpečný kód, ani nepomysleli.

Bezpečnost API tyto nástroje nezajistí. Dost často k tomu ani nejsou určené a už vůbec se na ně nemůžeme spoléhat. Bezpečnost zajišťuje celý vývojový tým - včetně vývojářů, projekťáků, byznys ownerů, testerů a dalších. Na to jsem se snažil upozornit právě v tomto e-booku. Návod, který jsem na těchto stránkách popsal, není platný pouze pro API. Je platný pro jakýkoliv software. Ať vyvíjíte webovou nebo desktopovou aplikaci. Ať používáte Node.js, Java, .NET nebo No-Code platformu, jako je třeba Make. Stále jde o vývoj software a stále bychom se měli zabývat výše zmíněnými body, i když v jistých obměnách.

Využijte sílu automatizace a automatizujte vše, co se dá. Jak jste se (doufám) dozvěděli napříč knihou, nedosáhnete jen výrazné finanční a časové optimalizace, ale taky rozvoje svých zaměstnanců, sehraných týmů, kvalitních dokumentací, zadání a dalších benefitů.

Chcete pomoct nebo se dozvědět víc?

Jsem **Štefan Prokop**, zakladatel projektu **Bezpečný kód**. Rád vám pomůžu s aplikační bezpečností. Podívejte se na služby, které nabízím.

- **Školím aplikační bezpečnost** vývojáře, týmy i management.
- **Dělám security code review** a mentoring vašich vývojářů nebo dodavatelů.
- **Dělám bezpečnostní audit** podle OWASP SAMM a **white-box pentest**.
- **Postarám se o aplikační bezpečnost** u vás ve firmě **od A do Z**.

Zaujalo vás to? Chcete pomoct s implementací kroků, které popisuju v tomto ebooku? **Napište mi na stefan.prokop@owasp.org**, nebo si nezávazně rezervujte termín schůzky.

Sledujte **Bezpečný kód** na

- LinkedInu,
- YouTube.



BEZPEČNÝ KÓD